



University of Tehran
College of Engineering
School of Electrical and Computer
Engineering



Replicated Key-Value Store

Eventual & Simplified Strong Consistency Models in Golang

Distributed Systems Assignment 3: CE Department, University of Tehran

June 2026

Table of Contents

1 Introduction & Overview	2
2 Theoretical Questions (Part 1)	2
2.1 Why is data replicated in distributed systems?	2
2.2 What is the difference between replication for performance and replication for fault tolerance?	2
2.3 What is the main consistency challenge in data replication?	2
2.4 What is the difference between Eventual Consistency and Strong Consistency?	3
2.5 What is the “Read-Your-Writes” consistency guarantee?	3
2.6 What is the “Monotonic Reads” consistency guarantee?	3
2.7 Why is it impossible to simultaneously achieve complete Consistency, Availability, and Partition Tolerance?	3
3 System Architecture & Design	3
3.1 Replica Nodes	3
3.2 HTTP API Design	3
3.3 Client CLI	4
3.4 Execution and Deployment	4
4 Consistency Models	5
4.1 Eventual Consistency	5
4.2 Simplified Strong Consistency	5
5 Versioning & Conflict Resolution	6
5.1 Data Structure	6
5.2 Replication Check & Last-Write-Wins (LWW)	6
6 Experimental Scenarios	7
6.1 Scenario 1: Temporary Inconsistency (Eventual Consistency)	7
6.2 Scenario 2: One Replica Failure	7
6.3 Scenario 3: Concurrent Conflict	7
6.4 Scenario 4: Network Delay Impact & Metrics	7
7 Discussion & Analysis	8
8 System Limitations & Future Work	8

1. Introduction & Overview

In distributed systems, data replication is utilized to improve system availability, durability, and read throughput. However, replicating data across multiple physical machines introduces the fundamental challenge of consistency management, as described by the CAP theorem.

This report presents the implementation and analysis of a versioned, replicated Key-Value store system consisting of three replica nodes. We implement two distinct consistency models:

- (a) **Eventual Consistency**: Highly available writes with background asynchronous replication.
- (b) **Simplified Strong Consistency**: Consistent reads and writes coordinated via replica majorities (quorums).

The system is evaluated across four distinct failure and performance scenarios to analyze response latency, convergence behavior, stale read counts, and conflict resolution.

2. Theoretical Questions (Part 1)

2.1 Why is data replicated in distributed systems?

Data is replicated in distributed systems for four key reasons:

- (a) **High Availability**: Storing data on multiple servers ensures that if one server fails, other replicas can immediately handle client requests.
- (b) **Fault Tolerance & Reliability**: Storing copies across separate hardware and physical locations prevents permanent loss of data in case of physical failure or disasters.
- (c) **Performance & Low Latency**: Placing replica nodes geographically closer to users reduces network latency (propagation delay) for local reads.
- (d) **Scalability & Read Throughput**: Spreading read operations across multiple replica nodes prevents any single server from becoming a CPU/IO bottleneck under high concurrent traffic.

2.2 What is the difference between replication for performance and replication for fault tolerance?

- **Replication for Performance**: Focuses on latency reduction and load sharing. Replicas are often placed close to users (e.g., in a CDN or regional caching layers). These systems often use asynchronous (eventual) replication to avoid write-blocking overhead, sacrificing strict consistency for speed and throughput.
- **Replication for Fault Tolerance**: Focuses on durability and consistency. Replicas are placed in separate failure domains (different power zones or regions). Updates are usually replicated synchronously using consensus algorithms (like Paxos or Raft) to ensure that if a subset of replicas fails, no committed data is lost. This values durability over low write latency.

2.3 What is the main consistency challenge in data replication?

The core challenge is the **propagation delay** of updates across replica nodes over a network. When data is updated at one replica, there is a delay before the update propagates to other replicas. During this window of inconsistency, different replicas store different versions of the data. If a client reads from a laggy replica, they will receive stale or conflicting data. Maintaining consistency requires coordination, which incurs latency, network traffic, and potential availability trade-offs.

2.4 What is the difference between Eventual Consistency and Strong Consistency?

- **Strong Consistency (Linearizability):** Guarantees that once a write operation returns successfully, all subsequent read operations by any client on any replica will return the new value. It makes the distributed system behave like a single node, at the cost of higher latency and lower write availability.
- **Eventual Consistency:** A weaker model guaranteeing that if no new updates are made to a key, all replicas will eventually converge to the same value. In the short term, clients may read stale data, but writes are fast and highly available because they do not block on synchronous propagation.

2.5 What is the “Read-Your-Writes” consistency guarantee?

It is a client-centric consistency model guaranteeing that a client will always see their own updates. If a client updates a key, any subsequent read of that key by the **same client** will return the updated value (or a newer one). It prevents the bad user experience where a user posts a comment, refreshes the page, and the comment is missing because the read query hit a stale replica.

2.6 What is the “Monotonic Reads” consistency guarantee?

Monotonic reads guarantee that if a client reads a certain value v of a key, they will never subsequently read a value older than v . If a client queries replica A and gets version 3, and then queries a laggy replica B, monotonic reads ensure the client does not get version 2 or 404 (stale read) — the system either redirects the query to an up-to-date node or blocks until replica B is synchronized.

2.7 Why is it impossible to simultaneously achieve complete Consistency, Availability, and Partition Tolerance?

As stated by the **CAP Theorem**, network partitions (P) are inevitable in real distributed systems. When a partition occurs and splits the system into isolated parts:

- To keep the system **Consistent (C)**, the reachable nodes must refuse write requests because they cannot synchronize them with the unreachable nodes. This makes the system **Unavailable (A)**.
- To keep the system **Available (A)**, the reachable nodes must accept write requests, but this causes the partitioned parts of the database to diverge, violating **Consistency (C)**.

Therefore, when a partition occurs, a system must choose between CP and AP.

3. System Architecture & Design

The system is implemented in Go (Golang) using only standard library packages for maximum performance, concurrency safety, and portability.

3.1 Replica Nodes

Each of the three replicas (`replica1`, `replica2`, `replica3`) runs as an independent OS process listening on HTTP ports 8001, 8002, and 8003.

- **Local Data Persistence:** Each replica maintains its state locally in memory using a map, which is safely serialized/deserialized to a local JSON file (`data_replica1.json`, etc.) upon updates.
- **Concurrent Safety:** Map reads and writes are guarded by a `sync.RWMutex` to prevent data races during concurrent read/write and replication requests.

3.2 HTTP API Design

Each replica exposes a lightweight HTTP JSON API:

- GET /get?key=k&consistency=c: Serves GET requests. Coordinates a quorum read if consistency is strong, or reads locally if eventual.
- GET /local-get?key=k: Internal endpoint. Returns the replica's local version of the key without coordination.
- POST /put: Serves PUT requests.

```
{ "key": "x", "value": "10", "consistency": "eventual" }
```

- POST /replicate: Internal endpoint. Receives replication payloads from peers and triggers local conflict resolution.
- POST /config: Dynamically updates replica configurations, such as simulating network transmission delay.

```
{ "delay_ms": 500 }
```

- POST /shutdown: Safely terminates the replica server process.
- GET /status: Returns a status report of the node configuration and current database state.

3.3 Client CLI

The client is implemented in `client/main.go` and supports two main operational modes:

- (a) **Manual Commands:** Send individual GET or PUT requests to any target replica URL.
- (b) **Automated Orchestrator:** Automatically builds, spawns, configs, monitors, and shuts down the replicas to execute scenarios 1–4 and record measurements cleanly.

3.4 Execution and Deployment

3.4.1 Compiling the Replicas

Before running the scenarios or manual tests, compile the replica server codebase into a binary inside the `client/` directory so the client orchestrator can execute it:

```
go build -o client/replica_bin replica/main.go
```

3.4.2 Running the Automated Scenarios

From the `client/` directory, you can run all automated scenarios sequentially to evaluate consistency characteristics and collect metrics:

```
go run main.go -run all
```

Or run individual scenarios:

```
go run main.go -run scenario1
go run main.go -run scenario2
go run main.go -run scenario3
go run main.go -run scenario4
```

3.4.3 Running Replicas and Client Manually

Alternatively, you can run the replicas as independent background processes using their JSON configurations:

```
./client/replica_bin -config configs/replica1.json  
./client/replica_bin -config configs/replica2.json  
./client/replica_bin -config configs/replica3.json
```

Then use the client CLI manual flags to perform individual operations:

```
# PUT operation with strong consistency  
go run main.go -op put -key mykey -value myval -replica http://localhost:8001 -  
consistency strong  
  
# GET operation with eventual consistency  
go run main.go -op get -key mykey -replica http://localhost:8002 -consistency  
eventual
```

4. Consistency Models

4.1 Eventual Consistency

Under eventual consistency, the system prioritizes availability over immediate consistency:

- **Write Path:** The coordinator replica receives the client PUT, writes the new value locally, increments the key's version number, and immediately returns success (HTTP 200) to the client. Parallel background goroutines are spawned to propagate the update via `/replicate` HTTP calls to peer replicas.
- **Read Path:** Client GET requests are answered immediately from the contacted replica's local database.
- **Trade-off:** Low write latency (acknowledgment does not block on network) but clients may read stale data if replication is in flight.

4.2 Simplified Strong Consistency

Strong consistency guarantees that once a write is acknowledged, all subsequent reads reflect that write:

- **Write Path (Read-Before-Write Quorum):**
 - (a) The coordinator queries all replicas in parallel via `/local-get` to determine the maximum current version number of the key.
 - (b) If at least a majority of replicas (2 out of 3) respond, the coordinator increments this maximum version (`max_version + 1`) and prepares the write.
 - (c) It sends the update via `/replicate` in parallel to all replicas.
 - (d) If at least a majority of replicas acknowledge successful write, the PUT returns success. Otherwise, it fails (HTTP 503).
- **Read Path (Quorum Read & Read Repair):**
 - (a) The coordinator queries itself and peers in parallel via `/local-get`.
 - (b) Once a majority (2 out of 3) responds, the coordinator chooses the entry with the highest version/timestamp.
 - (c) Stale replicas are asynchronously updated with the latest value (Read Repair).
 - (d) The latest value is returned to the client.

5. Versioning & Conflict Resolution

5.1 Data Structure

Every replica stores keys associated with their metadata to enable logical version tracking:

```
type KeyEntry struct {
    Key      string `json:"key"`
    Value     string `json:"value"`
    Version  int    `json:"version"`
    UpdatedBy string `json:"updated_by"`
    Timestamp int64  `json:"timestamp"` // Unix nano timestamp
}
```

5.2 Replication Check & Last-Write-Wins (LWW)

Upon receiving a replication update via `/replicate`, a replica performs deterministic conflict resolution:

- (a) If `received.version > local.version`, the local value is overwritten.
- (b) If `received.version < local.version`, the update is discarded.
- (c) If `received.version == local.version`, a write conflict is detected. The replica resolves it using LWW rules:
 - Overwrite if `received.timestamp > local.timestamp`.
 - Discard if `received.timestamp < local.timestamp`.
 - If timestamps are identical, tie-break by selecting the replica with the lexicographically higher ID (`received.updated_by > local.updated_by`).

6. Experimental Scenarios

All scenarios were executed programmatically using the automated orchestrator in the client CLI.

6.1 Scenario 1: Temporary Inconsistency (Eventual Consistency)

An artificial replication delay of 2000 ms is configured across the replicas. The client performs a PUT $x=10$ on replica1 under eventual consistency (succeeding in **538 μ s** with local version 1) and immediately queries replica2 for x .

- **Observed Result:** The immediate GET on replica2 returns a 404 Not Found error (server returned status 404: Key 'x' not found), demonstrating a stale read. After waiting 3 seconds (allowing the 2-second replication delay to elapse), a subsequent GET on replica2 successfully returns the value 10 (version 1, updated by replica1), confirming convergence.
- **Explanation:** The coordinator acknowledges the write immediately after updating its local state. The peer replication propagates asynchronously, leaving a 2000 ms inconsistency window during which reads on peers receive stale/empty data.

6.2 Scenario 2: One Replica Failure

Replica 3 is terminated to simulate node failure. The client performs writes on Replica 1:

- **Eventual Consistency:** PUT $y=\text{eventual_val}$ succeeds immediately (latency: **1.41 ms**) because eventual writes only require a local write confirmation.
- **Strong Consistency (2 Replicas Active):** PUT $z=\text{strong_val}$ succeeds (latency: **3.47 ms**) because 2 out of 3 replicas are active, satisfying the majority quorum size of 2. A subsequent strong GET z from Replica 2 succeeds.
- **Strong Consistency (1 Replica Active):** After stopping Replica 2 as well, leaving only Replica 1 active, a strong PUT $z2=\text{strong_val2}$ fails with HTTP 503 (Quorum write check failed: only 1 of 2 replicas available to get version) because majority quorum cannot be established. Eventual writes still succeed (latency: **455 μ s**).

6.3 Scenario 3: Concurrent Conflict

An artificial delay of 2000 ms is configured. The client concurrently sends PUT $c=\text{val_from_r1}$ to Replica 1 and PUT $c=\text{val_from_r2}$ to Replica 2 (delayed by 50 ms to ensure a logical/physical timestamp skew). Both writes initially create version 1 locally.

- **Immediate Reads:** Replica 1 returns val_from_r1 , Replica 2 returns val_from_r2 , and Replica 3 returns a blank value (no entry found yet).
- **Final Convergence:** After 3 seconds, all three replicas converge to val_from_r2 (updated by replica2).
- **Explanation:** Conflict resolution uses LWW. Since Replica 2's write occurred 50 ms later, it has a higher nanosecond timestamp and deterministically overwrites the update from Replica 1 on all nodes when the replication packets eventually arrive.

6.4 Scenario 4: Network Delay Impact & Metrics

Artificial network delays of 0 ms, 500 ms, and 2000 ms are injected. For each delay, both consistency models are tested. The latency, convergence time, and stale read counts are summarized below.

Delay (ms)	Model	PUT Latency (ms)	GET Latency (ms)	Convergence (ms)	Stale Reads
------------	-------	------------------	------------------	------------------	-------------

0 ms	Eventual	0	0	52	1
0 ms	Strong	2	0	0	0
500 ms	Eventual	0	0	514	10
500 ms	Strong	504	0	0	0
2000 ms	Eventual	1	0	2046	40
2000 ms	Strong	2004	0	0	0

7. Discussion & Analysis

- (a) **PUT Latency:** Eventual consistency PUT latency remains independent of network delay (always 1ms) because writes are asynchronous. Strong consistency PUT latency increases linearly with network delay because the coordinator must block and wait for majority acknowledgments.
- (b) **Read Stale Count:** Under eventual consistency, the number of stale reads observed scales linearly with network delay because the inconsistency window is prolonged. Strong consistency maintains zero stale reads regardless of delay due to synchronous majority reads and writes.
- (c) **CAP Theorem Alignment:** Eventual consistency chooses **AP** (Availability & Partition Tolerance) by allowing local writes during splits, while Strong consistency chooses **CP** (Consistency & Partition Tolerance) by rejecting writes when a majority cannot be reached.

8. System Limitations & Future Work

- **Physical Clock Drifts:** Last-Write-Wins (LWW) is vulnerable to wall-clock synchronization issues across machines. In production, logical clocks (Lamport, Vector Clocks) or Hybrid Logical Clocks (HLC) should be integrated.
- **Static Membership:** Adding or removing replica nodes requires manual config changes. A membership protocol like Raft or Gossip/SWIM would automate configuration updates.
- **Lack of Write Rollback:** If a strong consistency write fails halfway (e.g. only 1 replica confirms before others crash), the partially written state remains in the replicas. Implementing a two-phase commit (2PC) or a consensus-replicated log (Raft) is necessary to ensure strict atomicity.